

P16

Sistemas agentic contemporáneos: memoria, reflexión, multiagente e interoperabilidad

Sistemas agentic contemporáneos (nodo de frontera, revisable)

Varios (nodo compuesto) · 2023 · Conjunto de trabajos posteriores a ReAct — releer con fecha de consulta

Ficha pedagógica del Artificial Intelligence Evolution Program. No es el paper original: es una guía de lectura en español que enlaza a la fuente primaria. Consultado 2026-08-16.

P16 — Sistemas agentic contemporáneos

Nodo de frontera, revisable. No es un paper: es un conjunto de trabajos posteriores a ReAct que convierten el bucle en un sistema. Se lee con fecha de consulta y se relee.

Nivel: L5 · **Motor:** `agentic` · **Notebook:** `P16_agentic_systems.ipynb`

[!WARNING] Esta ficha es la más volátil del eje y se aparta deliberadamente del formato de las quince anteriores en un punto: **agrupa varios trabajos** en lugar de analizar uno. Lo hace explícito para no fingir un consenso que no existe. Lo estable aquí son las **preguntas**; lo inestable son los nombres de framework de cada temporada. Última revisión: **2026-08-16**.

1. Identificación

Campo	Valor
Título original	— (nodo compuesto, no es un paper único)
Autoría	Varios equipos independientes
Año	2023 en adelante
Venue	arXiv, NeurIPS, ICLR, UIST y especificaciones abiertas
Fuentes primarias	ver sección 18
Acceso	Abierto
Fecha de consulta	2026-08-16

Trabajos que componen el nodo

Trabajo	Año	Qué añade al bucle
Reflexion (Shinn et al.)	2023	Autocrítica verbal y memoria entre intentos
Generative Agents (Park et al.)	2023	Memoria episódica con recuperación por relevancia, importancia y recencia
Voyager (Wang et al.)	2023	Currículo autónomo y biblioteca de habilidades reutilizables
AutoGen (Wu et al.)	2023	Orquestación conversacional entre varios agentes
Model Context Protocol	2024	Estandarización del acceso a herramientas, recursos y prompts

2. Problema anterior

ReAct demostró el bucle y Toolformer demostró que el uso de herramientas se aprende. Pero un bucle desnudo no sobrevive al contacto con una tarea real:

- **no recuerda** nada de un episodio al siguiente;

- **no se corrige**: si el plan es malo, lo ejecuta hasta el final;
- **no tiene presupuesto**: puede consumir indefinidamente;
- **no sabe parar**: ante un fallo de herramienta, reintenta;
- **no escala a varios agentes** sin un protocolo de coordinación;
- **no tiene una forma estándar** de descubrir e invocar herramientas.

Cada uno de estos huecos generó una línea de trabajo.

3. Propuesta

No hay una propuesta única. Hay una **descomposición en componentes** que la práctica ha ido estabilizando, aunque los nombres varíen:

Componente	Función	Origen representativo
Plan	Descomponer el objetivo en pasos verificables	ReAct, Voyager
Herramientas tipadas	Acciones con contrato de entrada, salida y efectos	Toolformer, MCP
Memoria	Episódica, semántica y de trabajo, con recuperación	Generative Agents
Reflexión	Criticar el propio resultado y reintentar mejor	Reflexion
Presupuesto	Límite de pasos, tokens, coste y tiempo	práctica operativa
Criterio de parada	Cuándo terminar, abortar o escalar	práctica operativa
Orquestación	Varios agentes con roles y protocolo	AutoGen
Permisos y aislamiento	Qué puede tocar el agente y con qué autoridad	seguridad de sistemas

4. Intuición sin fórmulas

Un becario brillante sin instrucciones, sin presupuesto y sin nadie a quien preguntar cuando algo falla. El problema no es su capacidad: es que el **sistema** a su alrededor no existe. Un agente que funciona en la demo y falla en producción casi nunca falla por el modelo.

Dónde deja de funcionar la analogía: el becario sabe cuándo está fuera de su competencia y pregunta. El agente no tiene ese sentido; hay que construirlo explícitamente como criterio de parada y escalamiento.

5. Matemática mínima

No hay una ecuación central. Lo que hay es un **contrato operativo** que sí se puede formalizar:

```
estado_t = (objetivo, plan, memoria_t, presupuesto_restante_t)
```

```
a_t ~  $\pi(\cdot \mid \text{estado}_t)$ 
```

```
o_t = entorno(a_t) ← puede fallar
```

```
memoria_{t+1} = actualizar(memoria_t, a_t, o_t)
```

```
presupuesto_{t+1} = presupuesto_t - coste(a_t)
```

PARAR si: objetivo cumplido

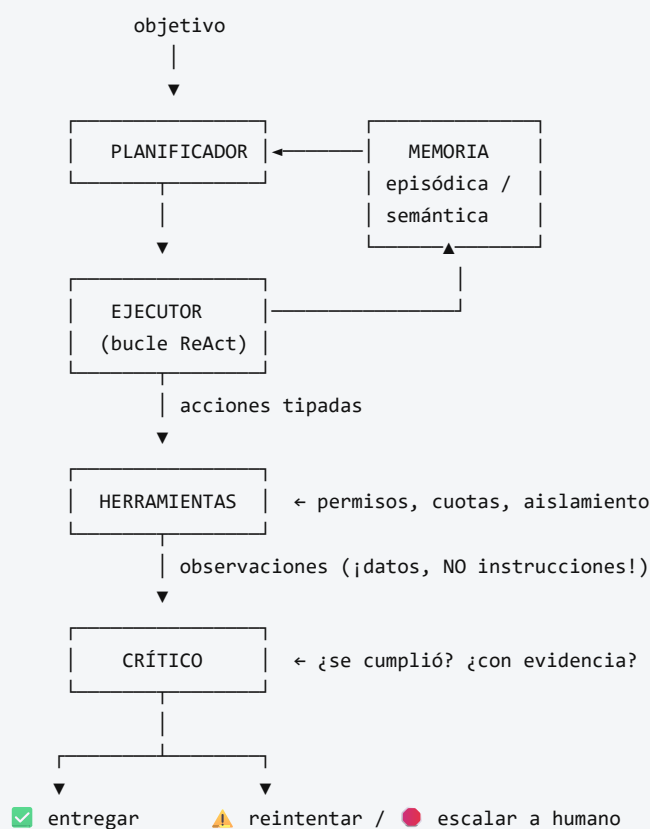
✓ presupuesto agotado

✓ fallo no recuperable → escalar a humano

✓ estado repetido → detectar bucle

La última línea es la que separa un prototipo de un sistema operable.

6. Arquitectura o flujo



Todo el bucle bajo PRESUPUESTO y con TRAZA persistida.

7. Qué observar en el paper original

Al ser un nodo compuesto, lo que hay que observar es **transversal**:

- En **Reflexion**: cómo se convierte un fallo en texto que mejora el siguiente intento, y cuál es el límite de esa mejora.
- En **Generative Agents**: la función de recuperación de memoria (relevancia, recencia, importancia) y la evaluación con humanos, no solo con benchmarks.

- En **Voyager**: la biblioteca de habilidades como forma de memoria **procedimental**, y la generación del currículo.
- En **AutoGen**: qué problemas mejoran con varios agentes y cuáles **no** — la sección más útil y la que menos se cita.
- En **MCP**: la separación entre *tools*, *resources* y *prompts*, y qué implica para permisos.
- **En todos**: cuántas ejecuciones se reportan y con qué varianza. Es la pregunta que más demos derriba.

8. Evidencia y resultados

Aquí hay que ser especialmente cuidadoso. El área tiene una brecha grande entre demostración y evidencia:

- muchos resultados se reportan sobre **pocas ejecuciones**, sin varianza;
- los **benchmarks de agentes** son jóvenes y algunos han mostrado problemas de contaminación o de especificación ambigua;
- la comparación entre frameworks rara vez controla el modelo base, el prompt y el presupuesto;
- el **coste** (llamadas, tokens, latencia) se omite con frecuencia, y es la mitad de la decisión.

Antes de citar cualquier cifra de esta área: comprobar número de ejecuciones, varianza, modelo base, presupuesto y fecha. Si falta alguno, la cifra no es comparable.

La miniatura de este eje no mide capacidad: muestra un agente con presupuesto explícito que se topa con un fallo de herramienta y **escala en lugar de responder igualmente**. Parar es un resultado correcto.

9. Impacto

- Traslada el foco del **prompt** al **sistema**: memoria, presupuesto, permisos, trazas y criterios de parada.
- Convierte la operación de agentes en una disciplina con su propio nombre y sus propias métricas (tasa de éxito, de escalamiento correcto, de respuesta inventada, coste por tarea).
- Hace de la **seguridad** un requisito de diseño: la inyección de prompt indirecta a través de observaciones es un vector real, no teórico.
- Empuja hacia la **estandarización** del acceso a herramientas, con las implicaciones de cadena de suministro que eso conlleva.

10. Limitaciones

1. **Evidencia débil y heterogénea.** Es la limitación principal de todo el nodo.
2. **Sin definición consensuada de «agente».** Cada trabajo usa la palabra para algo distinto.
3. **Fiabilidad compuesta.** Con 90 % de éxito por paso, diez pasos dan ≈ 35 %. La aritmética es implacable y se ignora demasiado a menudo.
4. **Coste y latencia** frecuentemente ausentes del reporte.
5. **Superficie de ataque amplia:** inyección indirecta, herramientas maliciosas, exfiltración de datos a través de acciones legítimas.
6. **Atribución de responsabilidad sin resolver:** quién responde cuando el agente actúa mal.

7. **Los nombres cambian más rápido que las ideas.** Buena parte de la literatura de una temporada queda obsoleta en la siguiente sin haber sido refutada: simplemente se abandona.

11. Errores comunes

Error	Corrección
«Funcionó una vez, está listo»	<code>n=1</code> no es evidencia. Sin distribución, varianza y casos adversarios, es una anécdota.
«Más agentes = mejor»	Añadir agentes multiplica coste, latencia y modos de fallo. Hay que demostrar que aporta.
«El agente es autónomo»	Opera bajo un objetivo, un presupuesto y unos permisos que alguien definió. La autonomía es un rango, no un interruptor.
«La traza demuestra cómo razonó»	Igual que en ReAct: es texto generado, útil para auditar decisiones, no para probar procesos.
«Si el agente responde, hizo su trabajo»	Un sistema que siempre responde está ocultando sus fallos. La abstención es una capacidad.
«El contenido que lee el agente son instrucciones»	Toda observación es dato . Tratarla como instrucción es la vulnerabilidad de inyección indirecta.
«Este es un campo maduro»	Es un campo activo con evidencia joven. Tratarlo como maduro es el error más caro.

12. Relación con trabajos anteriores

- **P13 ReAct (2022)** — el bucle base.
- **P14 Toolformer (2023)** — herramientas aprendidas.
- **P15 DPO (2023)** — alineación accesible del componente de decisión.
- **P11 RAG (2020)** — memoria consultable, antecedente directo de la memoria semántica de un agente.
- **Agentes racionales clásicos** (Russell y Norvig) — la definición de agente como percepción → decisión → acción es muy anterior a los LLM. Ver la clase 004 del programa.

13. Relación con trabajos posteriores

Por definición, esta sección **caduca**. Lo posterior a este nodo no se añade aquí: se registra con fecha y fuente en `frontier/current-topics.yaml`, y solo asciende a `papers/foundational/` cuando se consolida.

Preguntas abiertas que conviene seguir:

- evaluación de agentes con protocolos comparables y coste incluido;
- fiabilidad compuesta: cómo llevar tareas de muchos pasos a tasas de éxito operables;
- defensa contra inyección indirecta con garantías, no con heurísticas;
- memoria a largo plazo que no degrade con el volumen;
- atribución de responsabilidad y trazas verificables por terceros.

14. Notebook asociado

Qué implementa: un agente con plan, herramientas, memoria, presupuesto y criterio de parada que se topa con un fallo de verificación y escala en lugar de reintentar; el mapa componente → riesgo si falta; y el contraste entre un reporte anecdótico ($n=1$) y uno con distribución.

Qué NO implementa: ningún modelo de lenguaje. El agente **ejecuta** un plan, no lo planifica. Un agente real planifica con un LLM y puede equivocarse justo ahí.

```
ai-evolution paper-lab P16 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Enumera los seis componentes del contrato operativo y qué falla si quitas cada uno.
Explicar	Explica la fiabilidad compuesta con un ejemplo numérico de 10 pasos.
Aplicar	Reduce el presupuesto en el notebook y comprueba que el agente aborta limpiamente.
Analizar	Toma un trabajo de la tabla del nodo y analiza qué evidencia aporta y qué deja sin demostrar.
Evaluar	Te presentan un agente con «92 % de éxito». Escribe las siete preguntas que harías antes de aceptarlo.
Crear	Diseña el protocolo de evaluación de un agente de tu dominio: métricas, casos adversarios, presupuesto y criterio de aceptación.

16. Autoevaluación

1. ¿Qué distingue un bucle de un sistema agentic?
2. Con 95 % de éxito por paso, ¿cuál es la tasa esperada en una tarea de 15 pasos?
3. ¿Por qué la abstención es una capacidad y no un fallo?
4. ¿Qué es la inyección de prompt indirecta y qué regla la previene?
5. ¿Por qué añadir agentes puede empeorar un sistema?
6. ¿Qué debe contener el reporte de evaluación de un agente para ser aceptable?
7. ¿Por qué esta ficha lleva fecha de consulta destacada y las anteriores no tanto?

17. Respuestas esperadas

1. Los componentes que rodean al bucle: memoria, presupuesto, criterio de parada, permisos, trazas y escalamiento. El bucle es el motor; el sistema es el vehículo.
2. $0,95^{15} \approx 0,46$. Menos de la mitad. Es el argumento para acortar cadenas, verificar pasos intermedios y diseñar puntos de control.
3. Porque un sistema que siempre responde convierte sus fallos en respuestas plausibles. Poder decir «no lo sé» o «necesito ayuda» es lo que hace operable un sistema en producción.
4. Que contenido leído por el agente (una página, un documento, un correo) contenga texto dirigido a él. La regla: **toda observación es dato, nunca instrucción**; las instrucciones solo vienen del usuario por su canal.

5. Porque cada agente añade coste, latencia, puntos de fallo y ambigüedad de coordinación. El beneficio debe demostrarse contra una línea base de un solo agente bien construido.
6. Número de ejecuciones, varianza, tasa de éxito, de escalamiento correcto y de respuesta inventada, coste medio y percentil alto, modelo base, presupuesto, casos adversarios probados y fecha.
7. Porque agrupa trabajos recientes cuya evidencia aún se está consolidando. Las quince fichas anteriores describen resultados asentados y replicados; esta describe un frente activo.

18. Fuentes primarias

- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K. y Yao, S. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. **NeurIPS 2023**. [arXiv:2303.11366](https://arxiv.org/abs/2303.11366) · consultado 2026-08-16.
- Park, J. S. et al. (2023). *Generative Agents: Interactive Simulacra of Human Behavior*. **UIST 2023**. [arXiv:2304.03442](https://arxiv.org/abs/2304.03442) · consultado 2026-08-16.
- Wang, G. et al. (2023). *Voyager: An Open-Ended Embodied Agent with Large Language Models*. [arXiv:2305.16291](https://arxiv.org/abs/2305.16291) · consultado 2026-08-16.
- Wu, Q. et al. (2023). *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. [arXiv:2308.08155](https://arxiv.org/abs/2308.08155) · consultado 2026-08-16.
- *Model Context Protocol* — especificación abierta. modelcontextprotocol.io · consultado 2026-08-16.



Evaluación — P16 · Sistemas agentic contemporáneos: memoria, reflexión, multiagente e interoperabilidad

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Sistemas agentic contemporáneos (nodo de frontera, revisable)* (2023, Conjunto de trabajos posteriores a ReAct — releer con fecha de consulta) · **Nivel:** L5

Parte A — Contexto histórico (20 pts)

- (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
- (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

- (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
- (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

- (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

- (10) Ejecuta `P16_agentic_systems.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
- (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
- (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

- (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
- (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).